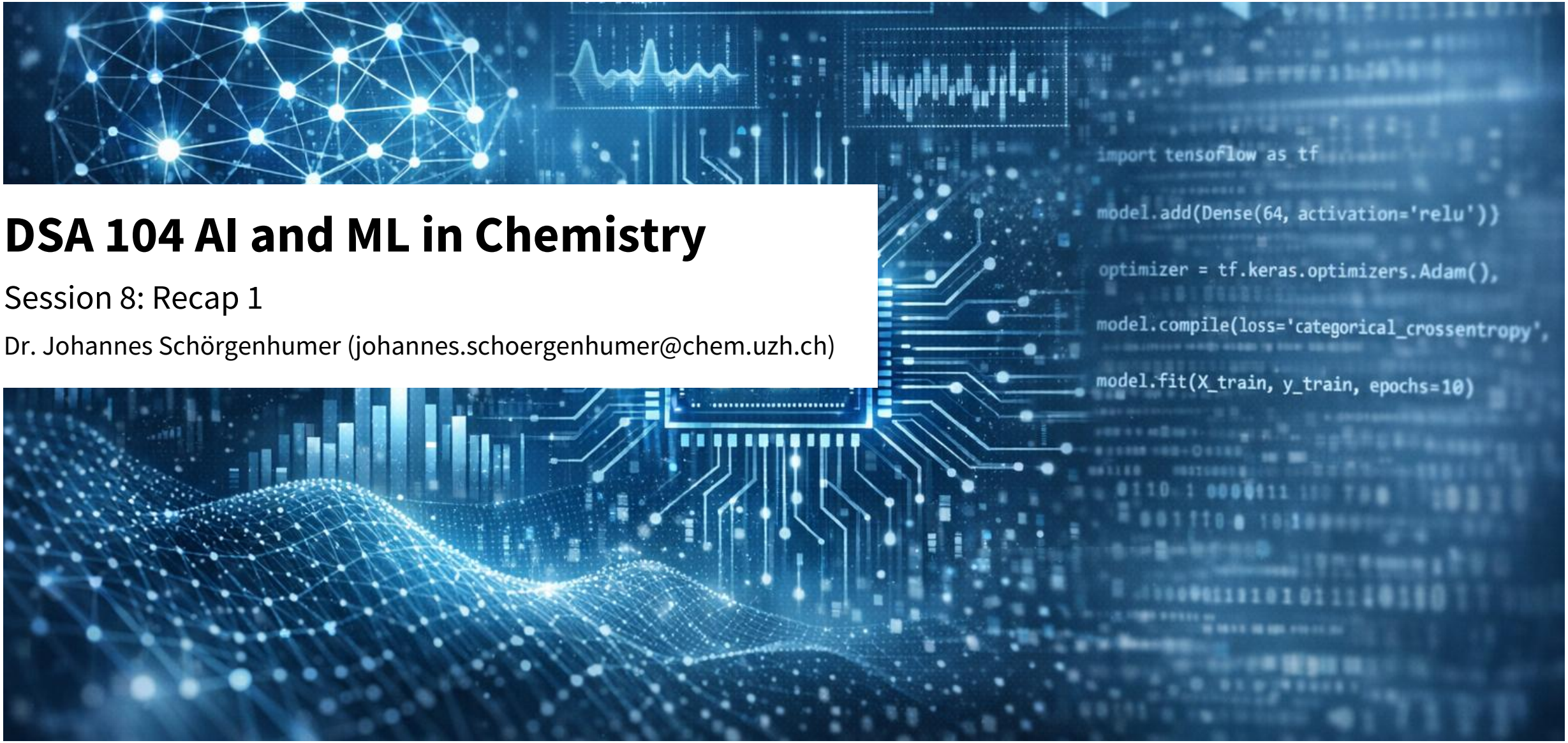
Universität Zürich[UZH]

Department of Chemistry



DSA 104 AI and ML in Chemistry

Session 8: Recap 1

Dr. Johannes Schörgenhumer (johannes.schoergenhumer@chem.uzh.ch)

Points for today

Discuss Assignment 2

Q&A, Feedback

Train&Test Splits
Pipelines

Quiz time

Light content: Time series

Q&A, Feedback

@Exercises:

— Useful in this format?

— Time demand?

— Technical issues?

Other feedback?

Points for recap session?

Better risk estimation - train/test split

Test Set Method

— Assumption: Samples are **independently and identically distributed**

— Split data set of *n* samples into **two non-overlapping subsets**: **Prevent data leakage!!**

  — **Training set**: for ERM (i.e. to optimise parameters), typically 80% of *n*

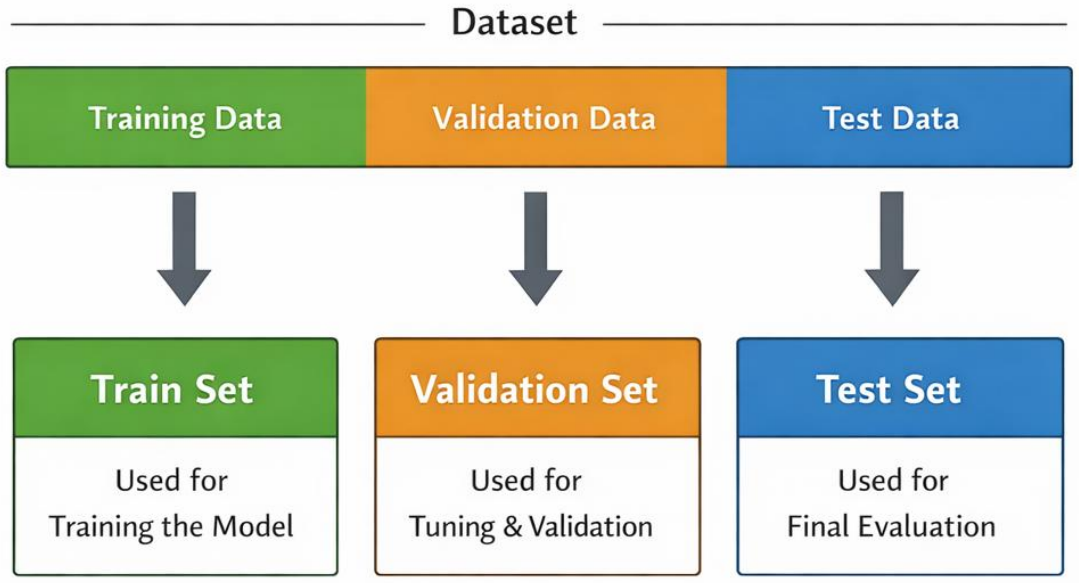
  — **Test set**: use to estimate the risk (test data = approximation of future, unseen data), typically 20% of *n*

— RE on the test set identifies overfitting!

Even better: train/validate/test split

Validation Set

- Hyperparameters often changed, model will be retrained and evaluated (**hyperparameter tuning**, e.g. CV!)
- May lead to overfitting:
  - Hyperparameter tuning based on test set evaluation results → Test set indirectly used for training
  - Not non-overlapping subsets - estimation of the generalisation error compromised
- Solution: Create a third non-overlapping **validation set**.
- Use the **validation set for hyperparameter tuning** and the **test set (once) for the final model evaluation**.



Validation: Cross-validation

Cross-validation (CV) = model evaluation to deliver more **stable and reliable estimation** of the generalisation:

- A single train/test split might not represent the true data distribution
- Performance may vary due to randomness
- Small datasets are particularly sensitive
- Working principle – e.g. K-fold CV:
  - Split the dataset into **K equal-sized folds (=subsets)**
  - Train the model on K-1 folds, test on the remaining one
  - Permutate and average the K-results
- Best practice: keep another independent subset for final model evaluation!

Train	Train	Train	Train	Test
Train	Train	Train	Test	Train
Train	Train	Test	Train	Train
Train	Test	Train	Train	Train
Test	Train	Train	Train	Train

Alexander Elvers – own work, CC BY-SA 4.0, https://commons.wikimedia.org/w/index.php?curid=84255403

Validation: Splitting Strategies

— Train/Test/Validation

— Cross validation: e.g. k-fold CV

Train	Train	Train	Train	Test
Train	Train	Train	Test	Train
Train	Train	Test	Train	Train
Train	Test	Train	Train	Train
Test	Train	Train	Train	Train

Fold = random split of shuffled data!

9	10	2	3	6	14	1	5	8	11	15	12	7	4	13

Other important CV methods:

— **Stratified k-fold**: response values close to equal in folds

— **Repeated k-fold**: repeat random splitting into k folds to average performance

Exhaustive CV methods: all possible splits explored

— **Leave one out (LOOCV)**: C_1^n (binomial coefficient)

— **Leave p out (LpOCV)**, e.g. p = 2 as almost unbiased method (but e.g. $C_2^{15} = \binom{15}{2} = 105$ possibilities…)

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

⋮

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Computational Statistics & Data Analysis, **2011**, *55*, 1828-1844, https://doi.org/10.1016/j.csda.2010.11.018.

Pipelines – hand in hand with CV!

- Pipelines guarantee that every preprocessing step is applied **correctly and consistently**
- Automatically **choose the right operation** for each step in the pipeline in training and prediction (fit_transform & fit, transform & predict)

```
pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("pca", PCA(n_components=5)),
    ("rf", RandomForestClassifier(random_state=42))
])

pipeline.fit(X_train, y_train)

y_pred = pipeline.predict(X_test)
```

- **Prevent data leakage**
  - Accidental use of test set in training
  - E.g. Scale or PCA fit to entire dataset
- **Clean structure**: Keep processing and model together
  - Scattered code segments, e.g. for scaler, pca, model (error-prone!)
  - With pipeline: one object!
- **Essential to CV** (would be rather arduous to do otherwise)

ML Quiz 1 – two truths and a lie:

— Mean Squared Error (MSE) is commonly used for regression tasks.

— Cross-entropy loss is widely used for classification tasks.

— Loss functions are only used during model evaluation, not training.

ML Quiz 1 – two truths and a lie:

— K-means finds the optimal number of clusters automatically.

— K-means clustering tries to minimize the distance between points and their cluster centroids.

— DBSCAN can detect clusters of arbitrary shape.

ML Quiz 1 – two truths and a lie:

— Decision trees generally produce smooth, continuous prediction boundaries.

— Decision trees can handle both numerical and categorical data.

— Decision trees are prone to overfitting without constraints.

ML Quiz 1 – two truths and a lie:

— PCA reduces dimensionality by projecting data onto directions of maximum variance.

— PCA works best when features are categorical.

— t-SNE is often used for visualizing high-dimensional data in 2D or 3D.

ML Quiz 1 – two truths and a lie:

— L1 regularization encourages sparsity in model weights.

— Regularization is only useful for small models with few parameters.

— L2 regularization penalizes large weights more strongly than small ones.

ML Quiz 2

Why is data normalization often necessary?

What is overfitting, and how can it be prevented?

What metrics can be used to evaluate classification models?

ML Quiz 2

What is the purpose of one-hot encoding?

Explain the bias–variance tradeoff in machine learning.

What is a confusion matrix?

ML Quiz 2

What is an outlier, and how can it affect a model?

What does a loss function represent?

What is a support vector machine (SVM)?

ML Quiz 2

What are decision trees, and what are their main advantages?

What is the difference between classification and regression?

What is the difference between Supervised and Unsupervised ML? What are difficulties especially associated to the latter?

Time Series

A sequence of observations ordered in time, e.g. sensor data, website traffic, …: $\boldsymbol{x_1}, \boldsymbol{x_2}, \boldsymbol{x_3}, \boldsymbol{x_4}, \dots \boldsymbol{x_t}$

Key characteristic:

— **Temporal dependency**: past observations influence future values

Typical goals:

— Forecasting

— Anomaly detection

— Classification of sequences

Time Series – key characteristics

Key properties:

— **Trend**: Long term increase/decrease

— **Seasonality**: Repeating patterns in regular intervals

— **Cycles**: Longer irrecular fluctuations

— **Noise**: Random variation not explained by the signal

Model: $y_t = Trend + Seasonality + Noise$

Stationarity for a time series is given if…

— Mean is constant

— Variance is constant

— Autocorrelation structure does not change

Typical transformations:

— Differencing

— Detrending ($y'_t = y_t - y_{t-1}$)

— Log transformations (e.g. FT)

— Seasonal decomposition

Time Series – how to treat the data:

Python datetime objects may be very useful!

Feature engineering:

— Lag features: past observations used as predictors ($x_{t-1}, x_{t-2}, x_{t-3}$)

— Rolling statistics: e.g. rolling mean ($rolling\ mean_t = \frac{1}{k}\sum_{i=0}^{k-1} x_{t-i}$)

— Calendar features: e.g. hour, day of the month

Data splits:

— Random splits don't work

— Use temporal slice instead!

Models:

— Classical models, e.g. AutoRegressive, Moving Average

— ML: Treat as regression!

— DL: NN, Transformers

First look at Time Series

Have a look at the provided notebook (*timeseries.ipynb*) and…

…discuss what the individual steps do.

…find some acceptable parameters

…how can you actually implement forecast of the next value? (Recursive vs. direct forecasting!)